



Design & Innovation Project (DIP)

Project Charter

Large Language Model Assisted Control Design and Auto Tuning for Underactuated Robotic Systems

Project Group: E026

**School of Electrical and Electronic Engineering
Academic Year 2026/27
Semester 1/2**

Date Prepared: 2026.9.3

Project Name:	Large Language Model Assisted Control Design and Auto Tuning for Underactuated Robotic Systems
Project Supervisor:	Professor Xie Lihua
Project Manager:	He Song Rui
Treasurer:	Zheng Yuxin

Project Purpose or Justification:

This project investigates the use of large language models (LLMs) as intelligent assistants for control design and auto-tuning in simple underactuated robotic systems, such as the inverted pendulum and two-wheel self-balancing robot. The project examines how an LLM can help generate initial control strategies, suggest parameter settings, and refine controller designs based on measured system performance.

Students will first build reproducible simulation environments and implement verified baseline controllers such as PID/LQR. The LLM will then be used to propose controller structures, tuning rules, reward formulations, or executable code from textual descriptions of the system and control objective. Expected outcomes include a benchmark simulation pipeline, baseline controller implementations, an LLM-assisted tuning framework, an RL pipeline, and a safety/validation layer for deployment to the STM32 platform. The project provides hands-on experience in control, robotics, simulation, and applied machine learning.

Project Description:

1. Overview

This project investigates how large language models (LLMs) can assist controller design, automatic tuning, and evaluation for an underactuated two-wheel self-balancing robot - an inherently unstable system in which body pitch and horizontal position must be regulated using wheel torques. The project uses a Yahboom STM32 robot platform together with a reproducible MuJoCo/Gymnasium simulation suite to develop and evaluate two AI-facilitated control methodologies: LLM-assisted PID/LQR tuning and LLM-assisted reinforcement learning (RL). Each methodology is evaluated through a non-harness/offline workflow and a harnessed hardware workflow, while preserving deterministic real-time control on the STM32.

2. Non-Harness Mode (Off-Board AI, Free Exploration)

In this mode, the LLM operates outside the robot using external computing resources and historical simulation or hardware logs. The goal is to explore controller design, tuning, and reward engineering without embedded compute constraints. For the PID/LQR approach, the LLM analyses closed-loop performance metrics and proposes bounded gain adjustments; each proposed update is validated in simulation before the next optimisation iteration proceeds. For the RL approach, the LLM analyses PPO training telemetry and proposes revisions to multi-component reward functions and skill definitions.

3. Harness Mode (LLM-Assisted Supervision with Safety Constraints)

In harness mode, the STM32 remains responsible for the deterministic, high-frequency control loop. The LLM operates asynchronously as a supervisory assistant: it analyses buffered performance summaries and proposes gain, target, mode, or reward updates, but it does not issue

Project Description:

high-frequency motor commands. No proposal is applied until it passes deterministic bounds checks, simulation or offline validation, and the project safety framework.

For the PID/LQR approach, the LLM acts as a supervisory tuner. For the RL approach, the LLM acts as an offline reward engineer and skill selector during development; the resulting compact policies are deployed as residual controllers around a verified PID baseline. The target real-time requirement remains a 200 Hz control loop, with the deployed MLP forward pass optimised to complete within 1.5 ms and the full sensor-to-output cycle within the 5 ms hardware budget.

4. Comparison of the Two Modes

Comparing the non-harness and harness workflows evaluates the trade-off between off-line exploration and safety-constrained adaptation. The comparison will determine whether improvements obtained through flexible off-board optimisation can be transferred to an online hardware workflow without compromising stability, latency, or validation discipline.

5. Methodology

The project follows a simulation-to-hardware methodology: reproduce and verify the baseline; develop both AI-assisted approaches in simulation; validate candidate controllers or policies through safety and transferability gates; deploy the optimised controllers and trained policies to the physical robot under supervision; then perform robustness testing, quantitative evaluation, statistical comparison, and iterative refinement.

6. Deliverables and Expected Outcome

The main deliverables are a reproducible simulation environment, an instrumented robot prototype, two operational control pipelines, a safety and validation framework, an experimental dataset, and a comparative evaluation. The final outcome will identify the relative strengths, limitations, and practical suitability of LLM-assisted PID/LQR tuning versus LLM-assisted learning-based control for embedded, resource-constrained robotic systems.

High-level Project and Product Requirements:

1. Hardware & Platform Specifications

- 1.1. **Target Platform:** The system shall be implemented on a Yahboom STM32 two-wheel self-balancing robot.
- 1.2. **Operating Objective (0-degree regulation):** The robot shall operate as an inverted pendulum, regulating its pitch angle to maintain a target 0-degree upright position.
- 1.3. **Resource Constraints:** All deployed control policies and algorithms shall satisfy the computational, memory, motor-output, and control-loop limits of the STM32 microcontroller.

2. Dynamic Performance & Convergence Limits

- 2.1. **Convergence & Transient Response:** Following a bounded initial disturbance (e.g. +/-10 degrees tilt), the control system shall return the pitch angle to the upright position with settling time < 1.5 s and maximum overshoot < 3 degrees.
- 2.2. **Steady-State Stability (Accounting for Hardware):** Recognising motor backlash and IMU noise, the steady-state RMS tilt error shall be maintained within a realistic band of +/-1.0 to +/-1.5 degrees from the 0-degree setpoint.
- 2.3. **Control Loop Latency:** The complete control cycle - including sensing, state estimation, baseline control, and RL residual execution - shall execute in < 5 ms, corresponding to a control frequency of at least 200 Hz.
- 2.4. **Baseline-Relative Performance:** All advanced controllers shall be evaluated against a verified manually tuned PID baseline and shall attempt to match or improve its steady-state error and motor PWM variance (chatter).
- 2.5. **RL Rapid Stabilisation (Pipeline B):** The LLM-assisted PPO policy shall be evaluated from extreme initial pitch angles in [-40 degrees, +40 degrees] on flat ground and for recovery after manual disturbances. Final time-to-upright and recovery targets shall be benchmarked against baseline characterisation.

3. Control Architecture & Dual Pipelines

Pipeline A: LLM-Assisted PID/LQR Tuning with Short-Window Adaptation

The classical-control approach uses the LLM to iteratively adjust proportional, integral, and derivative gains and, where applicable, LQR parameters through a bounded supervisory loop:

- 3.A.1. **Macro-Level Stabilisation:** The LLM analyses complete historical episodes (overshoot, settling time, steady-state error, control effort) to establish or refine baseline parameters.
- 3.A.2. **Short-Window Adaptation:** A moving 0.5- to 1.0-second telemetry window is summarised into error and rate features. The LLM proposes slower, bounded micro-adjustments to gains or targets to address sustained payload shifts or disturbances; the LLM itself is not inside the high-frequency motor loop.
- 3.A.3. **Safety Gate:** All metric-driven updates must pass parameter bounds, simulation validation, and hardware safety checks before deployment.

Pipeline B: LLM-Assisted Reward Design for RL with Harness-and-Skills Architecture

The learning-based approach positions the LLM as a reward engineer and supervisory designer for a library of discrete RL skills rather than a single unrestricted policy:

- 3.B.1. **Non-Harness Training:** The LLM processes off-board MuJoCo/Gymnasium PPO logs to refine multi-component reward structures for skills such as steady-state regulation, disturbance recovery, and extreme-angle stabilisation. Reward components include penalties for angle $|\theta|$, angular velocity $|\omega|$, acceleration $|\alpha|$, control force, and position, together with survival and disturbance-resilience bonuses. Component weights are normalised within $[0, 1]$, with $\sum(w_i) = 1$.
- 3.B.2. **Harness Execution:** Compact, pre-trained skill policies are deployed on-board as a residual control layer around the verified PID baseline. The controller monitors macro-state and selects or interpolates between validated skills. The LLM remains offline during hardware operation and issues no real-time motor commands or reward updates.
- 3.B.3. **Residual Correction:** The preferred deployment architecture is a small bounded residual torque correction on top of the baseline controller, providing a controlled interface between the learning policy and the physical robot.

4. LLM Integration & Role

- 4.1. **Supervisory & Offline Constraint:** The LLM shall function as an offline or asynchronous supervisory assistant. It shall not process the raw high-frequency sensor stream or generate high-frequency motor commands.
- 4.2. **Mode 1 Constraints (PID/LQR Tuning):** Proposed gain changes shall remain within predefined bounds and pass deterministic range checks and simulation validation before hardware use.
- 4.3. **Mode 2 Constraints (RL Reward Design):** The LLM shall ingest training logs and evaluation metrics to refine reward formulations, skill definitions, and experiment settings. The final RL network shall remain compact enough to meet the 5 ms overall control-loop budget.
- 4.4. **Architecture Selection:** The project considers direct short-window tuning, hierarchical supervision, harness-and-skills/tool invocation, residual correction, and offline-scientist workflows. The implemented system will use the safer supervisory/harness-and-skills and residual-correction pattern rather than direct LLM motor control.

5. Safety & Fallback Mechanisms

- 5.1. **Hardware Limits:** Enforce maximum motor-output (PWM) saturation caps and maximum allowable-angle thresholds (e.g. ± 40 degrees before abort).
- 5.2. **Automated Fallback:** A mathematically stable PID or LQR baseline shall remain available and automatically engage if unsafe behaviour, limit cycling, or constraint violations are detected.
- 5.3. **Emergency Protocols:** Include a hard software kill-switch to immediately cut motor power if the tilt exceeds unrecoverable limits.

Project Management plan:

1. Executive Summary

This project explores the integration of Large Language Models (LLMs) into the control-engineering workflow, specifically targeting the design and automatic tuning of controllers for underactuated robotic systems. The core hypothesis is that LLMs can interpret structured performance descriptions and experimental feedback to iteratively synthesise or optimise control parameters, reducing manual trial-and-error while maintaining or improving the performance of

conventional methods. The experimental testbed is the Yahboom SBR-STM32 two-wheeled self-balancing robot, selected as a low-cost, accessible, and reproducible platform.

2. Key Innovations and Sources of Opportunity

2.1 Key Innovations

2.1.1 LLM-Driven Control Design and Auto-Tuning Paradigm

LLMs are introduced into the control-parameter optimisation workflow for underactuated two-wheeled self-balancing robots, enabling the model to interpret structured performance descriptions and automatically generate or adjust PID/LQR parameters. Unlike conventional manual trial-and-error tuning, the approach uses an LLM as a constrained reasoning layer that proposes changes which are then validated by deterministic control and safety mechanisms.

2.1.2 Dual-Path Architecture: Harnessing and Off-Line

The project explores two complementary LLM intervention modes: (i) an asynchronous harnessing workflow in which the LLM analyses buffered metrics and proposes validated updates, and (ii) an off-line data-driven workflow in which the LLM performs batch tuning or reward redesign from historical operational logs. The two paths are interoperable: off-line experiments can generate safe initial parameters, while failures and edge cases from harnessing experiments can feed back into off-line analysis.

2.1.3 Engineering Validator for Physical Underactuated Hardware

A complete LLM-assisted validation platform is developed for the Yahboom SBR-STM32 robot, including a safety-verification module, data-abstraction layer, logging pipeline, and communication protocol stack. The architecture is intended to be reusable for subsequent experiments on related systems.

2.1.4 RL Rapid Stabilisation Performance

The LLM-assisted PPO policy shall be evaluated on its ability to stabilise the robot from extreme initial pitch angles in $[-40^\circ, +40^\circ]$ and to recover from disturbances during steady-state operation. Final performance targets will be benchmarked against baseline characterisation and refined before the final experiments.

2.2 Sources of Opportunity

Technological Maturity Inflection Point: The falling cost of LLM inference and improvements in code generation, data analysis, and reasoning make constrained control-engineering assistance increasingly practical.

Industrial Demand Gap: Conventional controller design relies heavily on expert knowledge and manual iterative tuning. AI-assisted design tools could reduce engineering effort, shorten tuning cycles, and improve adaptability.

Academic Research Gap: Existing work on LLM-assisted control is more mature for simulation and linear systems than for nonlinear, coupled, underactuated hardware. The project targets this underexplored deployment setting.

3. SMART Objectives

3.1 Objective 1: Baseline Controller Implementation

Specific: Implement and deploy a cascaded LQR/PID controller on the STM32 as the baseline.

Measurable: balance success rate $\geq 95\%$ over 60-second trials; maximum angular deviation $\leq 5^\circ$.

Achievable: based on Yahboom official materials and established control theory. **Relevant:** provides the baseline for all LLM-enhanced comparisons.

Time-bound: complete by M4 (Week 9).

Test Conditions: random start-up conditions; 60-second trials; external disturbances permitted; provisional success definition based on return to equilibrium within 5 seconds following a disturbance.

3.2 Objective 2: LLM Integration and Communication Pipeline

Specific: The LLM receives performance metrics through a communication interface, generates parameter or reward-update proposals, and passes safety verification.

Measurable: end-to-end communication success rate $\geq 99\%$; proposals passing safety verification $\geq 85\%$; average single iteration time ≤ 120 seconds.

Time-bound: complete by M5-M6 (Week 10).

Test Conditions: 100 simulation iterations with randomised initial conditions.

3.3 Objective 3A: Auto-Tuning Demonstration

Simulation Fidelity & Sim-to-Real Transferability: The MuJoCo/Gymnasium environment shall replicate relevant robot dynamics, including mass distribution, wheel/motor torque limits, IMU noise characteristics, and control-loop timing. Domain randomisation (mass, friction, battery voltage, sensor noise) shall be applied to reduce the sim-to-real gap, and a quantitative transferability check shall be completed before hardware deployment.

Specific: LLM-driven tuning demonstrates improvement over the baseline.

Measurable: settling-time reduction $\geq 30\%$; maximum-overshoot reduction $\geq 40\%$; steady-state-error reduction $\geq 50\%$. Time-bound: demonstrate by M7-M8 (Weeks 11-12), subject to baseline characterisation.

Test Scenarios: step response (velocity setpoint 0 $\rightarrow$ 0.5 m/s); pulse-disturbance response (0.5 N s pulse torque).

Measurement Method: 10 repeated trials per configuration, averaged with a unified metric-computation script.

3.4 Objective 3B: LLM-Assisted RL Rapid Stabilisation

Specific: An LLM-designed reward function enables a PPO agent to stabilise the inverted pendulum from extreme initial conditions faster than the PID baseline while preserving steady-state stability. The policy shall be evaluated from pitch angles in $[-40$ degrees, $+40$ degrees] and under manual disturbances.

Measurable: time-to-upright from ± 40 degrees ≤ 2.5 s (baseline PID ≤ 3 s); disturbance success rate $\geq 85\%$; steady-state RMS error ≤ 1.2 degrees; mean reward demonstrates stable convergence within 100K training steps.

Time-bound: complete by M7-M8 (Weeks 11-12).

Test Scenarios: (A) cold start at $+40$ degrees, zero angular velocity; (B) cold start at -40 degrees, zero angular velocity; (C) warm start at ± 40 degrees with ± 10 degrees/s angular velocity and manual disturbance at $t = 0.5$ s; (D) robustness under payload-induced centre-of-gravity shift.

Measurement Method: 10 independent trials per scenario; log $\theta(t)$, $\omega(t)$, and $u(t)$; compute time-to-upright, peak overshoot, settling time, IAE, and total variation of control.

3.5 Objective 4: Robustness Validation

Specific: The system maintains performance under varying payload configurations.

Measurable: success rate $\geq 80\%$ under at least 3 distinct payload configurations.

Achievable: systematic tests with incremental mass additions.

Relevant: demonstrates performance beyond nominal conditions.

Time-bound: complete by M8 (Week 11). Payload configurations: 50 g, 100 g, and 150 g mass blocks fixed at different positions on the chassis, producing natural centre-of-gravity shifts.

3.6 Objective 5: Final Report Delivery

Specific: Complete a comprehensive final project report.

Measurable: minimum 40 pages covering literature review, methodology, experimental design, results, discussion, and conclusions.

Achievable: systematic documentation throughout the project.

Relevant: primary project deliverable and knowledge-transfer mechanism.

Time-bound: complete by M9 (Week 12). The report shall include comparative experimental data for the harnessed versus off-line workflows and, if results warrant, the team may prepare an academic paper submission.

4. Meeting and Reporting Mechanisms

Weekly Team Meeting: Scheduled every Wednesday at 13:30. Each member reports individual progress, planned tasks, and technical bottlenecks. These sessions serve as the primary forum to align interface specifications between the hardware and software sub-teams.

Progress Blog: Maintained on a weekly basis. Members record work summaries, code commits, and detailed experiment logs to ensure full traceability and knowledge retention throughout the project lifecycle.

Summary Budget:

Proposed Project Budget

Itemized Budget Breakdown

No.	Budget Item and Included Components	Qty	Procurement Source	Budget (SGD)
1	Yahboom STM32 Self-Balancing Robot & Expansion Packages	1 set	Robot package / Expansion-package listing	\$220
2	12V 2200mAh Lithium Battery Pack & 2A Charger	1 set	Battery and charger listing	\$30
3	Programming and Communication Accessories	1 lot	Yahboom Official Tmall Store	\$40
4	Safety and Controlled-Test Materials	1 lot	Yahboom Official Tmall Store	\$50
5	Replacement and Maintenance Components	1 lot	Yahboom Official Tmall Store	\$60
6	Data-Logging and Measurement Accessories	1 lot	Yahboom Official Tmall Store	\$40

7	Shipping, Price Variation, and Contingency	1 allowance	Marketplace checkout	\$60
8	Cloud LLM API Credits and Token Usage	1 allowance	Direct API provider / Approved institutional billing	\$500
Total	Total Planned Budget	—	—	\$1,000

Budget Category Summary

Budget Category	Allocation (SGD)	Percentage (%)	Purpose & Scope
Robot Platform & Expansion Packages	\$220	22%	Main underactuated robot hardware setup
Batteries, Accessories, Spares & Contingency	\$280	28%	Peripherals (Items 3–6), power supply (Item 2), and shipping/contingency (Item 7)
Cloud LLM API Usage	\$500	50%	Model inference fees, prompt optimization, and evaluation
Total Project Budget	\$1000	100%	

Initial High-Level Risks:

Risk 1: Excessive LLM Inference Latency

Online LLM inference latency ($300\text{ ms} - 2\text{ s}$) substantially exceeds the 1 ms STM32 control period, threatening to block real-time closed-loop tuning and cause control instability or physical falls. **Mitigation:** The STM32 maintains continuous low-level real-time control via an **asynchronous architecture** while the LLM performs background optimization. Routine iterations utilize **local lightweight models** (Llama 3.1 8B via Ollama) for rapid inference, reserving GPT-4o for critical evaluation cycles. A **5-second timeout safeguard** skips delayed iterations to retain active parameters, supported by a **state caching mechanism** that reuses historical proposals for similar performance states.

Risk 2: Hardware Damage During Testing

Physical component failures, such as motor overloads, IMU degradation, or battery damage during testing, could halt physical experiments and lead to project schedule delays and budget overruns. **Mitigation:** An enforced **safety verification layer** checks all LLM-proposed parameters against gain bounds and maximum torque limits before flashing updates to hardware. Candidate parameters are deployed using a **soft-start protocol** scaled to 50% amplitude, while aggressive parameters undergo **simulation pre-screening** in MuJoCo/Gymnasium prior to physical execution. Additionally, dedicated **hardware reserves** are budgeted to enable rapid component replacement if damage occurs.

Risk 3: LLM Parameter Non-Convergence or Real-Test Failure

Parameter tuning iterations risk falling into unproductive loops without performance gains, wasting execution time, GPU runtime, and API credits without achieving control convergence. **Mitigation:** An **early stopping and rollback mechanism** automatically reverts the system to the best-known parameter set after N failed iterations. The workflow adopts a **decoupled, layered tuning strategy** that optimizes the outer velocity loop before tuning the inner angle loop. Furthermore, invalid or diverging parameters are filtered through **simulation pre-screening**, and a real-time **manual override interface** is provided to allow direct operator intervention when required.

Summary Milestones	Target Date
PC-Robot Communication System	Sept. 9
Baseline Controller Deployment and Validation	Sept. 16
Data Abstraction Layer (Feature Extractor) Development	Sept. 23
Harness Workflow Prototype	Sept. 23
LLM-off-line Path Initial Validation	Oct. 7
LLM-in-the-loop Path Initial Validation	Oct. 7
Dual-Path Comparative Experiment Design and Completion	Oct. 23
Robustness Validation Completion	Oct. 23
Final Report Writing and Delivery	Oct. 30

RACI Chart

RACI Chart	Person (In alphabetical order)					
Activity	Jiang Li	Jiayi	Songrui	Sridhar	Tze Jun	Yuxin
<i>Assemble and verify the balance bot hardware</i>	R	R	R	R	R	R
<i>Organize regular meetings and minute the meetings.</i>	I	I	R/A	I	I	I
<i>Experimental equipment purchasing and management</i>	I	I	A	I	I	I
<i>Developing a harnessed autotuning framework</i>	C	C	C	R	R	R
<i>Design and verify a non-harness, LLM assisted control system tuning workflow</i>	R	R	R	C	C	C

<i>Dual-Path Comparative Experiment Design and Completion</i>	R	R	R	A	R	R
<i>Final Report writing</i>	R	R	R	R	R	R